SLA-Adaptive Autoscaling for Cloud Applications Using Kubernetes

AWS Cloud Migration, End-to-End Architecture & Complete Implementation Blueprint

Project: SLA-Adaptive Kubernetes HPA on AWS	**Evaluation Target:** Web Microservices + Stateful Gaming (Minecraft)
Paper Reference: SLA-Adaptive Threshold Adjustment for K8s HPA	**Scope:** Real EKS/K3s Cluster, Prometheus PromQL, Locust Traffic
Team 10 Members: 23N217 Divya Nandhini | 23N243 Sai Karthi Balaji 23N264 Varalakshmi K | 24N435 Saran M	**Infrastructure Architecture:** AWS EKS (Option A) vs AWS EC2 K3s (Option B)

1. Executive Summary & Codebase Gap Analysis

Current State in Git Repository (SLA-CC): The current codebase is an academic prototype ('CloudScale Lab') written in FastAPI and React/TypeScript. It implements an in-memory simulation engine that models HPA replica growth and adaptive threshold shifts under predefined mathematical load shapes (ramp, spike, sawtooth). While the mathematical logic is verified, the prototype is purely simulated (mode: simulation) without real Kubernetes APIs, real Linux cgroups, or Prometheus metrics.

Objective of the AWS Implementation: Transition this simulation prototype into a fully functional cloud deployment on Amazon Web Services (AWS). An autonomous in-cluster Python controller daemon will query live Prometheus metrics to dynamically tune the Kubernetes Horizontal Pod Autoscaler (HPA) target thresholds, evaluated under realistic Locust load testing.

Dimension	Current Prototype (In Repo)	Target AWS Implementation (Next Phase)
Platform	Local in-memory synthetic state	AWS EKS or AWS EC2 Ubuntu K3s Cluster (Kubernetes v1.28+)
Telemetry	Math formula with seeded noise	Live Prometheus server scraping application & node cgroups via PromQL
Workload	None (pure synthetic calculations)	Real containerized HTTP Microservice + Stateful Minecraft Server
Autoscaling	Calculated within simulation loop	Active K8s HPA patched dynamically via Python Kubernetes API Client
Traffic Injection	Static array iteration	Distributed Locust on AWS EC2 executing dynamic load test scenarios
Dashboard	Local browser chart (Recharts)	CloudScale Lab Web UI + Grafana Dashboards backed by live AWS metrics

2. AWS Cloud Architecture Deployment Models

Attribute	Option A: AWS EKS (Managed Enterprise)	Option B: AWS EC2 Multi-Node K3s (Recommended Budget)
Control Plane	Managed AWS EKS Cluster (v1.28)	Lightweight K3s server on 1x EC2 t3.medium instance
Worker Compute	EKS Managed Node Group (2x t3.medium)	1x EC2 t3.medium worker node running pods
Load Generator	1x EC2 t3.small instance running Locust	1x EC2 t3.small instance outside the cluster running Locust
Storage & Ingress	AWS ALB + AWS EBS CSI Driver (gp3)	K3s Traefik / NGINX Ingress + Local-Path / EBS CSI volume
Estimated Cost	~$135 / month ($73 EKS fee + EC2 + NAT)	**~$25 - $35 / month** (or ~$12/mo on Spot / Free Tier). No EKS fee.

Recommendation: Option B (EC2 K3s) delivers 100% API compatibility with standard Kubernetes HPA at 75% lower cost, making it ideal for academic evaluation while preserving 1-click migration to EKS.

3. SLA-Adaptive Controller Formulation & Feedback Loop

Standard Kubernetes HPA relies on fixed CPU targets: desiredReplicas = ceil(currentReplicas * (currentCPU / targetCPU)). Under non-linear latency increases, fixed targets trigger scaling too late, causing severe SLO violations. Our adaptive algorithm computes a dynamic target threshold $T_{cpu}(t)$ at each control cycle $\Delta t = 10s$ based on observed P95 latency $R_{p95}(t)$ relative to the Service Level Objective (SLO):

Observed Prometheus Latency	Operating Regime	Adaptive Action Applied to Kubernetes HPA Spec
$R_{p95}(t) \geq SLO \times 1.5$	**CRITICAL VIOLATION**	$T_{cpu}(t+1) = max(T_{min}, T_{cpu}(t) - 1.5 \times \delta_{down})$ Aggressively lowers threshold to force immediate pod scale-out.
$SLO \times 1.2 \leq R_{p95}(t) < SLO \times 1.5$	**WARNING BUFFER**	$T_{cpu}(t+1) = max(T_{min}, T_{cpu}(t) - \delta_{down})$ Proactively lowers threshold before SLA violation occurs.
$0.8 \times SLO \leq R_{p95}(t) < SLO \times 1.2$	**STABLE REGION**	$T_{cpu}(t+1) = T_{cpu}(t)$ Holds current threshold; prevents controller thrashing.
$R_{p95}(t) < SLO \times 0.8$	**HEALTHY HEADROOM**	$T_{cpu}(t+1) = min(T_{max}, T_{cpu}(t) + \delta_{up})$ Conservatively raises threshold to consolidate pods and save cost.

Asymmetric Safety: Downward adjustments (δ_{down} = 5% to 7.5%) act 5x faster than upward relaxation (δ_{up} = 1%), preventing premature scale-down. A 60-second stabilization cooldown window prevents rapid pod flapping.

4. Core Implementation Modules on AWS

4.1. Workloads (Web Microservice & Minecraft Server):

• *Stateless Web App:* FastAPI REST service exposing /workload (computes synthetic CPU load) and /metrics (Prometheus client). Configured with cpu: 100m, memory: 128Mi requests and cpu: 500m, memory: 256Mi limits.
• *Stateful Gaming (Minecraft):* Deployed as a Kubernetes StatefulSet with AWS EBS gp3 volume. Serves real Minecraft protocol traffic to evaluate co-located cluster resource contention under stateful conditions.

4.2. In-Cluster Metrics Pipeline:

Prometheus scrapes application metrics every 5 seconds. The controller uses the following PromQL queries:
• *P95 Latency:* histogram_quantile(0.95, sum(rate(http_request_duration_seconds_bucket[1m])) by (le)) * 1000
• *Pod CPU:* sum(rate(container_cpu_usage_seconds_total{container='webapp'}[1m])) / sum(kube_pod_container_resource_requests{resource='cpu'}) * 100

4.3. Autonomous Adaptive Controller Daemon (Python In-Cluster Operator):

Runs as a Kubernetes Deployment with RBAC permissions to patch HPA resources. Real operational code:

```
import time, requests
from kubernetes import client, config

config.load_incluster_config()
hpa_api = client.AutoscalingV2Api()
PROM = 'http://prometheus-server.monitoring.svc.cluster.local'

def run_control_cycle(slo_ms=200, current_target=70):
    query = 'histogram_quantile(0.95, sum(rate(http_request_duration_seconds_bucket[1m])) by (le)) * 1000'
    resp = requests.get(f'{PROM}/api/v1/query', params={'query': query}).json()
    p95 = float(resp['data']['result'][0]['value'][1])
    if p95 >= slo_ms * 1.5: current_target = max(40, current_target - 7.5)
    elif p95 >= slo_ms * 1.2: current_target = max(40, current_target - 5.0)
    elif p95 < slo_ms * 0.8: current_target = min(85, current_target + 1.0)
    body = [{'op': 'replace', 'path': '/spec/metrics/0/resource/target/averageUtilization', 'value': int(current_target)}]
    hpa_api.patch_namespaced_horizontal_pod_autoscaler('webapp-hpa', 'sla-autoscaling', body)
    return current_target
```

5. Scientific Evaluation & Comparative Experiments

To rigorously validate the adaptive autoscaler, paired test runs execute identical Locust load profiles across two configurations: (1) Conventional HPA with fixed 70% CPU threshold, and (2) SLA-Adaptive HPA.

Evaluation Metric	Formula / Measurement Method	Expected Research Finding
SLO Violation Rate (%)	*(Violated time steps / Total time steps) × 100*	> 50% reduction in SLA violations during flash crowds.
P95 Latency Degradation	Peak P95 latency recorded during workload transitions.	Peak latency remains constrained within the warning band.
Scaling Delay (Lag)	Seconds from traffic surge start until pod readiness.	Proactive threshold lowering reduces lag by 20-30 seconds.
Replica Efficiency & Cost	Average pod count & AWS EC2 instance hours consumed.	Higher threshold during idle periods lowers cloud infrastructure cost.
Controller Flapping	Count of scale-up / scale-down oscillating events.	Asymmetric damping prevents excessive churn.

6. Step-by-Step Implementation Roadmap (Phases 1 to 5)

Phase 1: AWS Infrastructure Provisioning (Days 1 - 2)

• Launch AWS VPC (2 public subnets, Internet Gateway, Security Groups for K8s ports 6443, 80, 443).
• Provision 2x t3.medium Ubuntu 22.04 EC2 instances (1 Master, 1 Worker) or 1 EKS cluster.
• Install K3s using curl -sfL https://get.k3s.io | sh - and install Kubernetes Metrics Server.
• Verify cluster health: kubectl get nodes -o wide.

Phase 2: Observability Stack Deployment (Days 3 - 4)

• Deploy Prometheus and Grafana via Helm: helm install prometheus prometheus-community/kube-prometheus-stack -n monitoring --create-namespace.
• Configure Prometheus ServiceMonitor to scrape the web app's /metrics endpoint every 5s.
• Create Grafana dashboards visualizing real-time Pod CPU, Memory, Request Rate, and Latency quantiles.

Phase 3: Workload Containerization & Deployment (Days 5 - 6)

• Build Docker image for the target web application instrumented with Prometheus client metrics.
• Push image to AWS ECR; deploy webapp-deployment.yaml and webapp-hpa.yaml in namespace sla-autoscaling.
• Deploy stateful Minecraft game server StatefulSet with AWS EBS gp3 volume for real-world background workload benchmarking.

Phase 4: Adaptive Controller Daemon Build & In-Cluster Deployment (Days 7 - 9)

• Refactor backend/app/autoscaler/adaptive_controller.py and backend/app/metrics/prometheus.py to execute against live Prometheus.
• Apply RBAC manifests (ClusterRole, ClusterRoleBinding, ServiceAccount) granting HPA patch permissions.
• Deploy the controller container to Kubernetes; verify active threshold patching with kubectl get hpa -w.

Phase 5: Locust Load Testing, Benchmarking & Research Report (Days 10 - 12)

• Deploy Locust on an external EC2 instance; execute Ramp, Spike, and Sawtooth test profiles against the application.
• Run Test Series A (Fixed 70% HPA) and Test Series B (SLA-Adaptive HPA) under identical traffic seeds.
• Export Prometheus telemetry to CSV; generate comparative graphs (SLO violations, P95 latency, replica efficiency).
• Connect CloudScale Lab frontend to the live AWS backend for live project demonstration.

Final Deliverables Checklist:
✓ **Running AWS Kubernetes Cluster** with live Prometheus & Grafana telemetry.
✓ **Adaptive Controller Daemon** actively modulating HPA thresholds based on live P95 latency.
✓ **Empirical Dataset (CSV/JSON)** comparing Fixed HPA vs SLA-Adaptive HPA across 4 workload patterns.
✓ **CloudScale Lab Web UI** integrated with live AWS metrics for interactive demonstration.
✓ **Final Academic Project Report** containing architecture diagrams, methodology, and results.